

Data Structures and Algorithms – IT1170

Practical 2 – Insertion Sort / Answers

Exercise 1

1. Write a Java program to implement Insertion Sort to sort an integer array in ascending order. Save the work as InsertionSort.java. Corresponding pseudocode is given below.

➤ Java Program for Ascending Order

```
import java.util.Arrays;

public class InsertionSort {

    public static void main(String[] args) {

        int[] arr = {8, 6, 1, 9, 4};

        for (int j = 1; j < arr.length; j++) {

            int key = arr[j];

            int i = j - 1;

            while (i >= 0 && arr[i] > key) {

                arr[i + 1] = arr[i];

                i--;

            }

            arr[i + 1] = key;

        }

        System.out.println("Sorted Array: " +
Arrays.toString(arr));

    }

}
```

➤ Pseudocode

BEGIN

FOR $j = 1$ **TO** $length-1$

$key = A[j]$

$i = j - 1$

WHILE $i \geq 0$ **AND** $A[i] > key$

$A[i+1] = A[i]$

$i = i - 1$

END WHILE

$A[i+1] = key$

END FOR

DISPLAY A

END

2. Modify the above insertion sort algorithm to sort the array in descending order.

➤ Algorithm (Descending Order)

→ **Step 1: Start**

→ **Step 2: Read the array**

→ **Step 3: For** $j = 1$ **to** $n-1$

$key = A[j]$

$i = j - 1$

While $i \geq 0$ **and** $A[i] < key$

$A[i+1] = A[i]$

$i = i - 1$

End While

```
        A[i+1] = key
→ Step 4: Display sorted array
→ Step 5: End
```

➤ Pseudocode (Descending Order)

BEGIN

FOR *j* = 1 **TO** *length-1*

key ← *A[j]*

i ← *j* - 1

WHILE *i* >= 0 **AND** *A[i]* < *key*

A[i+1] ← *A[i]*

i ← *i* - 1

END WHILE

A[i+1] ← *key*

END FOR

DISPLAY *A*

END

➤ Java Program (Descending Order)

```
import java.util.Arrays;
```

```
public class InsertionSortDescending {
```

```
    public static void main(String[] args) {
```

```
        int[] arr = {8, 6, 1, 9, 4};
```

```
        for (int j = 1; j < arr.length; j++) {
```

```
            int key = arr[j];
```

```
            int i = j - 1;
```

```

        while (i >= 0 && arr[i] < key) {

            arr[i + 1] = arr[i];
            i--;
        }

        arr[i + 1] = key;
    }

    System.out.println("Sorted Array (Descending): "
        + Arrays.toString(arr));
}
}

```

Exercise 2

3. Modify the program to count and display the number of shifts required during the sorting process.

```

import java.util.Arrays;

public class InsertionSort {

    public static void main(String[] args) {

        int[] arr = {8, 6, 1, 9, 4};

        int shifts = 0;

        for (int j = 1; j < arr.length; j++) {

            int key = arr[j];

            int i = j - 1;

            while (i >= 0 && arr[i] > key) {

                arr[i + 1] = arr[i];

                shifts++;

                i--;
            }

            arr[i + 1] = key;

        }

        System.out.println("Sorted Array: " +
            Arrays.toString(arr));
    }
}

```

```

        System.out.println("Total Shifts = " + shifts);
    }
}

```

Exercise 3

4. Implement a StepSimulation.java program that simulates the step-by-step execution of the Insertion Sort algorithm.

```

import java.util.Arrays;
import java.util.Scanner;

public class StepSimulation {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter number of elements: ");

        int n = input.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter elements:");

        for(int i=0;i<n;i++) {

            arr[i] = input.nextInt();

        }

        System.out.println("Initial List: "

            + Arrays.toString(arr));

        int step = 1;

        for(int j=1;j<n;j++) {

            int key = arr[j];

            int i = j - 1;

            while(i>=0 && arr[i] > key) {

                arr[i+1] = arr[i];

                i--;

            }

        }
    }
}

```

```
        arr[i+1] = key;

        System.out.println("Step "

            + step++

            + ": "

            + Arrays.toString(arr));
    }

    System.out.println("Sorted List: "

        + Arrays.toString(arr));

    input.close();
}

}
```